

Test Pattern Generation Using a Dual MAC Unit

Sung-il BAE, Sungchul YOON, Sangwook KIM and Sungho KANG*

Computer Systems & Reliable SoC Laboratory, Electrical & Electronic Engineering, Yonsei University, Seoul 120-749

(Received 23 April 2002)

BIST (built-in self test) is one of the most attractive solutions to test chips at speed. In this paper, we present a new test pattern generation method using the processing units of DSP (digital signal processing). This method utilizes dual MAC (multiply and accumulate) units, which consist of two multipliers and several accumulators, in DSP to generate test patterns. The proposed test pattern generator can generate two test patterns per cycle by using two multiplication processes and an accumulation process. One multiplier has a seed value while another multiplier performs a recursive multiplication. The accumulator performs the addition of two products in order to generate pseudo random test patterns. Consequently, either the two blocks can be tested simultaneously or a block can be tested by using various test-pattern combinations. The results on ISCAS (international symposium, on circuits and systems) benchmark circuits show that the new method can generate test patterns efficiently without LFSRs (linear feedback shift registers).

PACS numbers: 84.30.Bv

Keywords: BIST, Test pattern generation, LFSR, DSP, MAC

I. INTRODUCTION

In recent years, the rapid expansion of the consumer electronics market has resulted in a tremendous increase in the scope of applications for DSP (digital signal processing) systems. There is an increasing demand for high-performance DSP products which, in turn, necessitates structured DFT (design for testability) methodologies and, in particular, BIST (built-in self test) techniques. The conventional BIST schemes usually employ test pattern generators and test response compactors [1]. LFSRs (linear feedback shift registers) and MISRs (multiple input signature registers) are mainly used for the generation of test vectors and compaction of test responses [2], respectively. In the pattern-generation scheme, LFSRs have a simple hardware and comparatively good fault coverage. However, those conventional methods require an extra hardware overhead due to the LFSR.

Recently, it was proposed that a processing unit, such as a multiplier, an ALU (arithmetic logic unit) and an accumulator, be used as a test pattern generator and a test response compactor for BIST. The accumulator-based scheme modifies the addition and the carry propagation processes and the generation of random test patterns [3]. The ALU-based scheme proposes a new random number generation system and generates pseudo-random patterns [4,5].

The DSP consists of instruction fetch/decoding units,

a MAC (multiply and accumulate) unit, a RAM pointer unit, and so on. Among these fundamental units, the MAC unit, which performs various arithmetic functions, is the most important unit. The MAC unit has a multiplier, an adder, and an accumulator and performs a multiplication, an addition, a subtraction, and an accumulation at once. The research on the MAC-based scheme has just begun [6].

In this paper, we propose a new test pattern generation method. The proposed method uses on multiplication and on accumulation processes of dual MAC units and generates test patterns. The proposed scheme has a high fault coverage without LFSRs.

II. TEST PATTERN GENERATION

Most test schemes for digital circuits generate test patterns using a random pattern generator. This generation method has a relatively small hardware overhead with a high fault coverage. For stuck-at fault detection of digital circuits, a BIST scheme is considered. Figure 1 shows a conventional BIST scheme which consists of a random pattern generator, a response compactor and a signature analyzer [7]. In general, the BIST scheme, mentioned above, uses the random pattern generation method with an LFSR. An LFSR has many registers and modulo-2 units (a modulo-2 unit performs a logical arithmetic similar to an exclusive-OR) with a recursive data path.

Figure 2 shows an example illustrating the typical structure of an LFSR with 3 registers. At first, the LFSR

*E-mail: shkang@yonsei.ac.kr; Fax: +82-2-313-8053

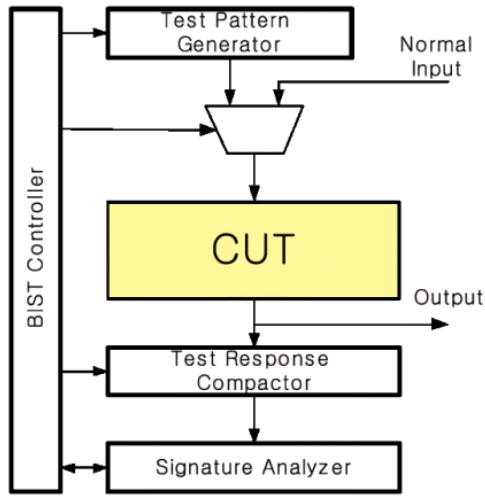


Fig. 1. Conventional BIST block diagram.

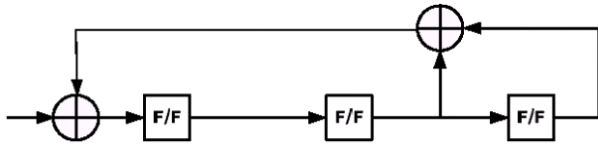


Fig. 2. LFSR structure.

is initialized. Commonly, one of the registers is set to a logical value '1', and the remaining registers are set to a logical value '0'. Then, the LFSR does a shift process per cycle with a modulo-2 arithmetic. This process is performed recursively with $2^n - 1$ period, where n is the number of flipflops. The BIST controller applies the test patterns generated by the LFSR to a test circuit, instead of the normal inputs. The test patterns are applied to the CUT (circuit under test). Test responses are generated according to each test pattern. If the CUT has a stuck-at fault, then the fault is propagated to output ports, and the test responses are different from our expectations. The test response compactor compresses test responses for effective memory usage. Finally, the signature analyzer decides whether or not the CUT has a stuck-at fault. An LFSR generates effective test patterns with a simple structure; however, it cannot achieve a high fault average. Lately, many reseeding techniques have been proposed in order to increase the efficiency of an LFSR [8].

In recent researches, many test pattern generation methods for DSPs have been proposed. These methods use the functional units of DSP to generate random test patterns. First, a new scheme using the accumulator as a pattern generator was proposed.

A test pattern generation method using an accumulator is shown in Fig. 3. The accumulator is initialized to zero. A seed is applied to one input of the adder, and it is added to the value of the accumulator. Then, the sum of the adder is saved to the accumulator again. The

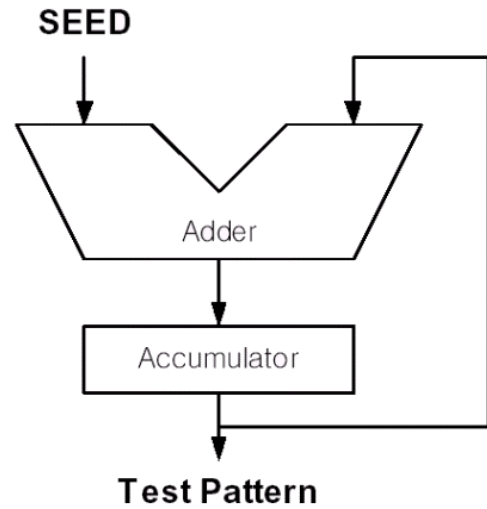


Fig. 3. Accumulator-based TPG structure.

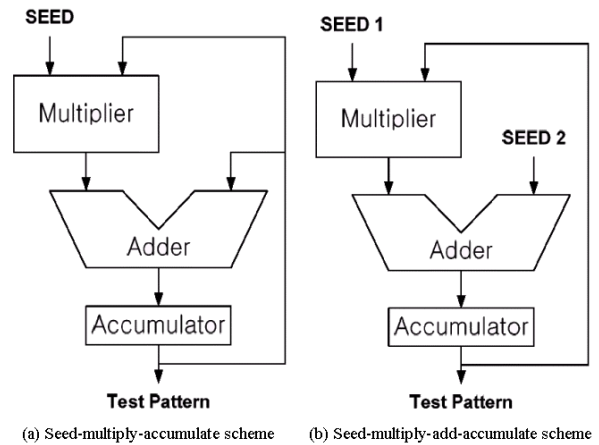


Fig. 4. MAC-based TPG structures.

value of the accumulator is recursively added to the seed as a new operand of the addition. Finally, this scheme generates a test pattern with a recursive addition.

Latest DSPs have a special unit called MAC. The MAC unit consists of a multiplier and an accumulator and performs multiplication and accumulation simultaneously. Some DSPs have two or more MAC units to increase the performance. As compared with the accumulator-based TPG method, the MAC-based TPG method uses an addition, an accumulation, and a multiplication. Through the multiplication, a greater variety of sequences of test vectors can be generated.

Figure 4 shows the structures of the main MAC-based TPG schemes in the case of a single MAC. Figure 4(a) shows a seed-multiply-accumulate scheme, and Figure 4(b) shows a seed-multiply-add-accumulate scheme. The seed-multiply-accumulate scheme is a basic method for a MAC-based TPG, and it has a simple generation process. The seed-multiply-add-accumulate scheme is a modified version of the seed-multiply-accumulate scheme. It is

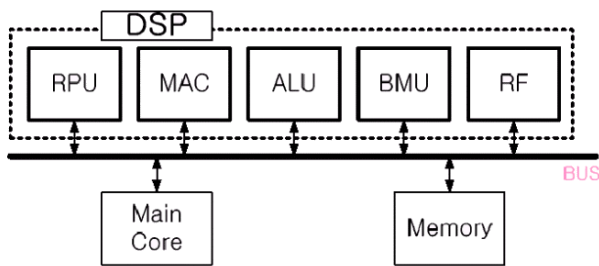


Fig. 5. Basic DSP architecture.

suitable for generating various test vector sets and re-seeding. These schemes use the functional units of the MAC-structured DSP effectively; however, they generate just one test pattern per cycle.

III. DUAL MAC UNIT-BASED TPG

In this research, two MAC units are used for a dual MAC DSP to generate the test patterns. In contrast to the accumulator-based TPG, the dual MAC unit-based TPG uses a multiplication process. The variation of the test vectors using the adders is small, but it is relatively large when the multiplication is used.

1. Dual MAC DSP Architecture

As a high processing power is required for digital signal, the importance of DSP is increased. Figure 5 shows the structure of a typical DSP with a single MAC unit. It has one multiplication and accumulation unit and performs a single multiplication and accumulation per cycle. The BMU (bit manipulation unit) performs a shifting and a logical arithmetic operations for the bit-field. The RPU (RAM point unit) calculates the addresses used by the other units. The RF (register file) consists of registers and controls their loading and storing.

Recently, a dual MAC DSP was developed to provide high performance. The dual MAC DSP has two multiplication units and generates two products per cycle. Therefore, the dual MAC DSP has two times the performance of a single MAC DSP. Figure 6 shows the structure of the dual MAC DSP. Each multiplier has a multiplicand and a multiplier, so various combinations of the multiplicand and the multiplier are possible. Moreover, the addition and the subtraction of generated products and accumulators are possible by using ALU. The other units of the dual MAC DSP have the same structures as with the single MAC DSP and perform the same processes.

In a brief explanation of the DSP process, the MAC unit mainly consists of two units: a multiplier and an accumulator. The multiplier takes two operands and

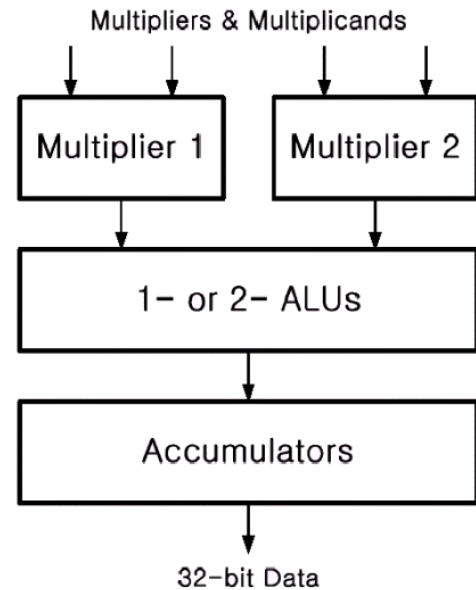


Fig. 6. DSP structure with dual MAC units.

multiplies them. The multiplication process generates a highly differential product, and in the dual MAC DSP, two products are generated at once. The ALU takes two or more operands and adds them. In general, the multiplier has an unsigned multiplication logic. After the multiplication, the product is sign-extended and shifted by the BFU (bit field unit). The two products generated by the dual MAC unit are accumulated. The accumulated value can be applied to two multiplier recursively. This relationship between the multiplier and the accumulator is important to digital signal processing fields since addition after multiplication is widely used in most signal processing algorithms.

2. Basic Random Number Generation Algorithm

Equation (1) shows the typical formula of random number generation for computer languages (for examples, C & C++ language):

$$x_{k+1} = x_k \times \text{seed} + \text{const} . \quad (1)$$

This recursive equation generates many pseudorandom numbers. A *seed* value must be commonly be an odd number since multiplication with an even number can not generate odd number. However, if Eq. (1) is applied to the DSP architecture, it generates a limited number of pseudorandom numbers. We assume a DSP architecture with a 16×16 multiplier and a 32-bit accumulator. The multiplier of the multiplier is *seed*, and an addend of the adder is constant. In the initial state, the multiplicand is zero, and it is multiplied by *seed*. The product after the multiplication is added to a constant. The sum is the first

test pattern, and it is stored in the accumulator. This stored value is applied to the multiplicand again. In this way, the MAC unit can generate random test patterns, but the number of test patterns using this method can not exceed about 15,000 patterns since the multiplicand has half the bit-length of the accumulator.

3. Modified Random Number Generation Algorithm

We modify a typical random number generation algorithm to achieve more than 15,000 test patterns. The new scheme is as follows: In the dual MAC DSP structure, multipliers and multiplicands have half the bit-length of the accumulators. This limitation makes the multiplicand seize half the data (upper or lower half). Also, the random-number generation algorithm cannot operate correctly. Therefore, we modify the multiplicand inputting process by using dual MAC units. In this method, the process up to the first accumulation is the same as it is in the previous algorithm, but the recursive process is different. First of all, we divide the accumulator by half. The upper half of the accumulator is applied to the multiplicand of one multiplier, and the lower half is applied to the multiplicand of another multiplier.

For various test patterns, the second multiplier calculates a square and the ALU adds the products generated in each multiplier. In this scheme, a *seed* value is not important any more. Simply, the *seed* value is an odd number. In the case that the seed value is an even number, the efficiency of the test pattern generation is degraded due to the generation of only even numbers at one multiplier. Regardless of the *seed* value, this method can stably generate more than 15,000 test patterns. Equation (2) explains the proposed scheme in brief:

$$\begin{aligned} \text{product1} &= \text{seed} \times \text{accumulator}[32 : 16] \\ \text{product2} &= \text{accumulator}[15 : 0] \times \text{accumulator}[15 : 0] \\ \text{accumulator} &= \text{product1} + \text{product2} \end{aligned} \quad (2)$$

Seed is a 16-bit value, and product1, product2, and accumulator are 32-bit values. This generation scheme provides the test patterns, which can detect more faults than the conventional method based on LFSRs.

4. Proposed Architecture

Figure 7 show the proposed test pattern generation scheme. Basically, the structure of the dual MAC is not changed. In addition, the new scheme needs 5 MUXs and a BIST controller. MUXs choose the data connections according to the operating mode.

In the normal mode, MUXs 'A' select normal inputs (X0, X1, Y0, Y1), and two multipliers multiply them.

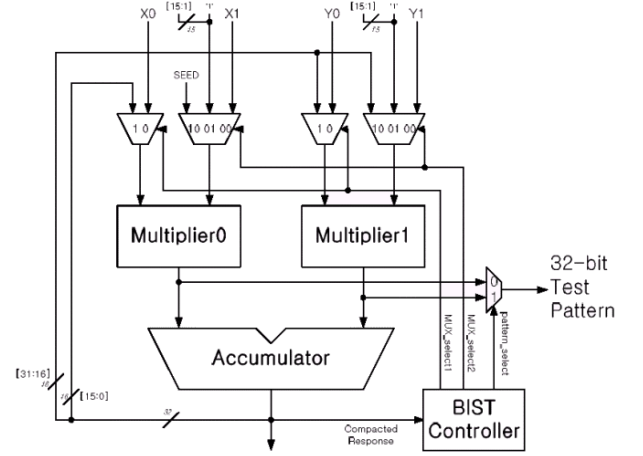


Fig. 7. Proposed TPG structure.

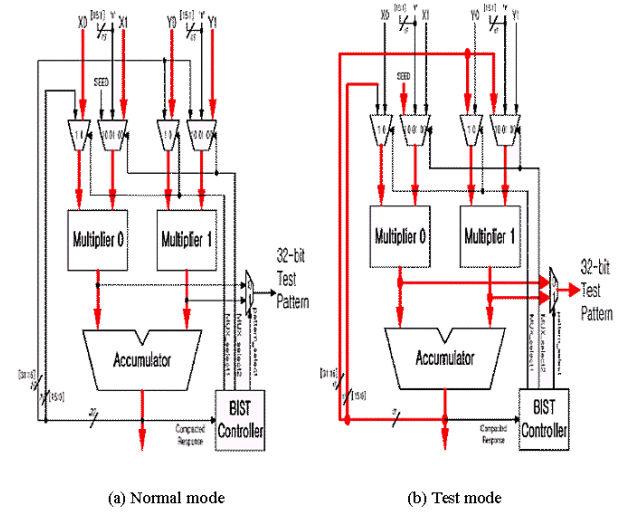


Fig. 8. Data flow of proposed scheme according to the mode.

Then, the accumulator with the ALU adds and stores the products. In the BIST mode, MUXs 'A' select the recursive inputs for the first multiplier and the same inputs for the second multiplier. One input of the multiplier is fed as a SEED value. MUX 'B' chooses the output test pattern by selecting one of two products or the final accumulator value. Figure 8(a) shows the data flow under the normal mode, and Fig. 8(b) shows the data flow under the test mode.

5. Three Test Pattern Extraction Schemes

There are three methods to extract test patterns. In the first method, we use the value of a 32-bit accumulator as a test pattern. This method is similar to the conventional method, and it can generate one test pattern per cycle.

Table 1. Fault coverage comparison I using the first method.

ISCAS85	(a) LFSR		(b) the first method			(c) # of pattern ≤ 10000		
	fault cov.	# of pat.	fault cov.	FC Diff.	# of pat.	LFSR	MAC	FC Diff.
c17	100	5	100	0	5	100	100	0
c95	95.45	355	95.45	0	390	95.45	95.45	0
c432	99.24	2413	99.24	0	2844	99.24	99.24	0
c499	98.94	3813	98.94	0	4059	98.94	98.94	0
c880	99.04	10020	99.36	0.32	6120	99.04	99.47	0.43
c1355	99.49	5699	99.49	0	5248	99.49	99.49	0
c1908	99.52	7953	99.47	-0.05	7656	99.52	99.47	-0.05
c2670	84.35	27261	84.64	0.29	29591	84.27	84.49	0.22
c3540	95.71	980	95.74	0.03	7950	95.71	95.83	0.12
c5315	98.9	31328	98.9	0	21894	98.77	98.88	0.11
c6288	99.56	2144	99.56	0	2240	99.56	99.56	0
c7552	94.91	89631	94.98	0.07	88182	94.93	92.42	-2.51
ISCAS89	(a) LFSR		(b) the first method			(c) # of pattern ≤ 100000		
	fault cov.	# of pat.	fault cov.	FC Diff.	# of pat.	LFSR	MAC	FC Diff.
cs27	100	7	100	0	21	100	100	0
cs208	99.53	3344	99.07	-0.46	2394	100	100	0
cs298	100	170	100	0	510	100	100	0
cs344	87.71	1584	87.71	0	1800	87.71	87.71	0
cs349	86.33	1584	86.33	0	1800	86.33	86.33	0
cs382	100	264	100	0	552	100	100	0
cs386	100	2600	100	0	1911	100	100	0
cs400	98.58	1800	98.58	0	2088	98.58	98.58	0
cs444	97.05	1920	97.05	0	1848	97.05	97.05	0
cs510	100	675	100	0	875	100	100	0
cs526	98.2	4392	99.28	1.08	6240	99.82	99.82	0
cs641	96.57	8046	97.86	1.29	7992	97.22	98.72	1.5
cs713	90.71	8046	91.74	1.03	7992	91.22	92.43	1.21
cs820	97.29	5267	99.29	2	8119	100	100	0
cs832	95.63	5267	97.7	2.07	8119	98.39	98.39	0
cs838	86.35	17085	86.23	-0.12	11591	88.45	87.75	-0.7
cs953	98.98	11655	98.98	0	13905	100	100	0
cs1196	97.42	6176	97.67	0.25	8768	99.92	99.92	0
cs1238	93.28	10304	92.99	-0.29	8768	94.83	94.83	0
cs1423	97.69	10374	98.68	0.99	26390	98.88	99.08	0.2
cs1488	99.73	3304	99.73	0	3080	100	100	0
cs1494	98.94	3304	98.94	0	3080	99.2	99.2	0
cs5378	98.15	75756	98.61	0.46	100152	98.33	98.61	0.28
cs9234	84.24	100035	84.32	0.08	100035	84.24	82.14	-2.1
cs15850	90.15	100204	90.57	0.42	100204	90.15	90.57	0.42
cs38417	87.95	101504	88.11	0.16	101504	87.95	88.11	0.16
cs38584	91.76	101016	92.1	0.34	101016	91.76	92.1	0.34

* The number of the test patterns is limited to about 100,000.

In the second method, we use the values of two 32-bit products and a 32-bit accumulator as test patterns. This method can generate two or more test patterns per

cycle, and it can test several blocks at the same time. Furthermore, we use the combination of two or more 32-bit test patterns per cycle to generate longer than 32-bit

Table 2. Fault coverage comparison II using the combinational mode.

ISCAS85	(a) LFSR		(b) the first method			(c) # of pattern ≤ 10000		
	fault cov.	# of pat.	fault cov.	FC Diff.	# of pat.	LFSR	MAC	FC Diff.
c17	100	5	100	0	11	100	100	0
c95	95.45	355	95.45	0	186	95.45	95.45	0
c432	99.24	2413	99.24	0	2089	99.24	99.24	0
c499	98.94	3813	98.94	0	3240	98.94	98.94	0
c880	99.04	10020	99.26	0.22	8341	99.04	99.36	0.32
c1355	99.49	5699	99.49	0	5823	99.49	99.49	0
c1908	99.52	7953	99.04	-0.48	5809	99.52	99.47	-0.05
c2670	84.35	27261	84.67	0.32	21903	84.27	84.67	0.4
c3540	95.71	980	95.86	0.15	15651	95.71	95.83	0.12
c5315	98.9	31328	98.9	0	30261	98.77	98.9	0.13
c6288	99.56	2144	99.56	0	1377	99.56	99.56	0
c7552	94.91	89631	94.57	-0.34	41194	94.93	95.38	0.45
ISCAS89	(a) LFSR		(b) the first method			(c) # of pattern ≤ 100000		
	fault cov.	# of pat.	fault cov.	FC Diff.	# of pat.	LFSR	MAC	FC Diff.
cs27	100	7	100	0	14	100	100	0
cs208	99.53	3344	100	0.47	3287	100	99.07	-0.93
cs298	100	170	100	0	340	100	100	0
cs344	87.71	1584	87.71	0	1440	87.71	87.71	0
cs349	86.33	1584	86.33	0	1440	86.33	86.33	0
cs382	100	264	100	0	696	100	100	0
cs386	100	2600	100	0	2210	100	99.74	-0.26
cs400	98.58	1800	98.58	0	2040	98.58	98.58	0
cs444	97.05	1920	97.05	0	2160	97.05	97.05	0
cs510	100	675	100	0	1200	100	100	0
cs526	98.2	4392	98.38	0.18	5088	99.82	97.66	-2.16
cs641	96.57	8046	97.86	1.29	10962	97.22	98.5	1.28
cs713	90.71	8046	91.74	1.03	10962	91.22	92.25	1.03
cs820	97.29	5267	97.29	0	7820	100	99.53	-0.47
cs832	95.63	5267	95.86	0.23	7820	98.39	97.93	-0.46
cs838	86.35	17085	87.28	0.93	11524	88.45	90.9	2.45
cs953	98.98	11655	97.5	-1.48	10755	100	98.15	-1.85
cs1196	97.42	6176	95.09	-2.33	10752	99.92	96.86	-3.06
cs1238	93.28	10304	92.1	-1.18	13216	94.83	92.25	-2.58
cs1423	97.69	10374	98.42	0.73	17472	98.88	98.88	0
cs1488	99.73	3304	100	0.27	3388	100	100	0
cs1494	98.94	3304	99.2	0.26	3780	99.2	99.2	0
cs5378	98.15	75756	98.02	-0.13	100152	98.33	98.02	-0.31
cs9234	84.24	100035	82.14	-2.1	100035	84.24	82.14	-2.1
cs15850	90.15	100204	90.3	0.15	100204	90.15	90.3	0.15
cs38417	87.95	101504	87.2	-0.75	101504	87.95	87.2	-0.75
cs38584	91.76	101016	90.86	-0.9	101016	91.76	90.86	-0.9

* The number of the test patterns is limited to about 100,000.

test patterns.

The third method is a hybrid of the previous two methods. This processing algorithm is as follows. When a cer-

tain number of consecutive test patterns of the first multiplier cannot detect any new fault, the BIST controller generates a signal to change the test pattern source or

the seed. The test pattern generation continues using the second multiplier or reseeding technique. Since extra memory (seed ROM) is required for the reseeding technique, the test pattern is changed.

IV. PERFORMANCE EVALUATION

In order to prove the effectiveness of the proposed scheme, we have performed a fault simulation on the ISCAS'85 and the ISCAS'89 benchmark circuits. First, we simulated one output test pattern (the first method) with the specific fault non-detectable period. We compared the fault coverage of the test patterns generated by LFSRs and by the first method using the dual MAC. The fault simulation was terminated when 1024 consecutive test patterns could no longer detect a fault. Tables 1(a) and (b) show that the new scheme is effective for the ISCAS'85 and the ISCAS'89 benchmark circuits. For the ISCAS'85 benchmark circuits, the proposed scheme has a higher fault coverage with fewer test patterns for all circuits except two. In the cases of the c1908 and the c7552 benchmark circuits, the simulation program terminated early because of the characteristic of the generated test pattern sequence. For the ISCAS'89 benchmark circuit, the proposed scheme has a fault coverage similar to that of the conventional method using LFSRs. Although some of the circuits show a little bit less fault coverage, the fault coverage degradation is very small. For most circuits, the test patterns obtained by using the first method provide a proper fault coverage compared to the conventional method using LFSRs. For a better comparison, Table 1(c) shows a fault coverage comparison when the same number of test patterns is generated. In the results, test patterns generated by the first method using the dual MAC can effectively detect stuck-at faults.

Table 2 shows the simulation results of the combination mode of the second and the third methods. When specific consecutive test patterns of one multiplier cannot detect a fault any more, the test patterns of another multiplier are used. The fault coverage differences between the two methods are mostly very small. Also, Table 2 shows that the proposed method is very effective.

In the results for the ISCAS'85 and the ISCAS'89 benchmark circuits, the proposed scheme can generate test pattern sequences without fault coverage degradation compared with the conventional method using an LFSR. Especially, for the ISCAS'85 benchmark circuit, the proposed scheme is more effective.

Table 3 summarizes the performance of the proposed scheme compared with the conventional LFSR scheme. The fault coverage of the proposed scheme is 1 % higher than that of the conventional scheme on average. However, it no longer needs the pattern generation unit as an LFSR. This scheme can be applied to various other processors composed of mathematical units. In these applications, the proposed scheme provides an optimized

Table 3. Performance comparison.

	Conventional Scheme (32-bit LFSRs)	Proposed Scheme
Fault Coverage	1.00	1.01
Hardware Overhead	1.00	0.25

structure for the specific pattern generation by using an internal unit with a lower additional hardware overhead compared with the conventional scheme. The hardware overhead of the proposed scheme is a quarter of the overhead of the conventional scheme.

V. CONCLUSIONS

All processors and circuits have self-test circuits. However, the conventional self-test circuits, such as BIST using LFSRs, have an effective fault coverage, but require an additional hardware. In this paper, we propose a dual MAC to be used as a pseudo-random test-pattern generator. The results for the ISCAS'85 and the ISCAS'89 benchmark circuits show that the dual MAC is suitable for test-pattern generation and can achieve a high fault coverage. In addition, the new method requires a smaller hardware overhead than the conventional method.

For a future work, various seeding multiplication approaches using the dual MAC should be investigated in order to achieve better fault coverage. Also, a compaction scheme using a dual MAC should be investigated.

ACKNOWLEDGMENTS

This research was supported by Samsung Electronics Co., Ltd.

REFERENCES

- [1] V. Agrawal, Ch. Kime and K. Saluja, IEEE Design and Test of Computers **10**, 73 (1993).
- [2] P. Bardell, W. McAnney and J. Savir, *Built-In Test for VLSI, Pseudorandom Techniques* (John Wiley Sons, Oct., 1987).
- [3] N. Saxena & J. Robinson, IEEE Trans. Comput. **35**, 317 (1986).
- [4] S. Adhamt, M. Kassab, N. Mukherjee, K. Radecka, J. Rajskit, and J. Tyszer, Proceedings of the IEEE Custom Integrated Circuits Conference **29**, 659 (1995).
- [5] J. Rajski and J. Tyszer, in *Proceedings of the Third Asian Test Symposium* (Nara, Nov., 1994), p. 227.
- [6] Albercht P. Stroele, in *Proceedings of the 16th IEEE VLSI Test Symposium* (Monterey, April, 1998), p. 78.

- [7] Xiaoling Sun, Michael Olson, and Daniel Yeung, in *Proceedings of the IEEE Pacific RIM Conference on Communications, Computer, and Signal Processing* (Victoria, May, 1995), p. 457.
- [8] E. Kalligeros, X. Kavousianos, D. Bakalis, and D. Niko-
los, in *Proceedings of the Seventh International On-Line
Testing Workshop* (Taormina, July, 2001), p. 80.